

```
In [1]: #Churn:Leave
import pandas as pd          #Step 1:Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
```

```
In [3]: df=pd.read_csv('C:/Users/hp/Desktop/customer_churn.csv')    #Step 2:Load Dataset
print(df.head())
```

	Customer ID	Gender	Age	MonthlyCharges	Tenure	Contract	Churn
0	1	Male	49	109.20	18	One year	No
1	2	Female	56	83.11	9	Month-to-month	No
2	3	Male	66	99.48	58	One year	No
3	4	Male	69	70.26	17	One year	No
4	5	Male	49	77.69	7	Month-to-month	No

```
In [5]: print(df.info())          #Step 3:Basic Info check
print(df.describe())
print(df['churn'].value_counts())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer ID           200 non-null   int64
1   Gender                 200 non-null   object
2   Age                    200 non-null   int64
3   MonthlyCharges         200 non-null   float64
4   Tenure                 200 non-null   int64
5   Contract               200 non-null   object
6   Churn                  200 non-null   object
dtypes: float64(1), int64(3), object(3)
memory usage: 11.1+ KB
None
```

	Customer ID	Age	MonthlyCharges	Tenure
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	44.405000	71.658100	34.755000
std	57.879185	15.353816	29.423599	20.798999
min	1.000000	18.000000	21.080000	1.000000
25%	50.750000	32.750000	48.537500	17.000000
50%	100.500000	46.000000	74.160000	34.000000
75%	150.250000	56.000000	97.160000	53.000000
max	200.000000	69.000000	119.050000	71.000000

```

-----
KeyError                                Traceback (most recent call last)
File ~\AppData\Roaming\Python\Python313\site-packages\pandas\core\indexes\base.p
y:3805, in Index.get_loc(self, key)
    3804 try:
-> 3805     return self._engine.get_loc(casted_key)
    3806 except KeyError as err:

File index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas\_libs\hashtable_class_helper.pxi:7081, in pandas._libs.hashtable.Py
ObjectHashTable.get_item()

File pandas\_libs\hashtable_class_helper.pxi:7089, in pandas._libs.hashtable.Py
ObjectHashTable.get_item()

KeyError: 'churn'

The above exception was the direct cause of the following exception:

KeyError                                Traceback (most recent call last)
Cell In[5], line 3
      1 print(df.info())
      2 print(df.describe())
----> 3 print(df['churn'].value_counts())

File ~\AppData\Roaming\Python\Python313\site-packages\pandas\core\frame.py:4102,
in DataFrame._getitem__(self, key)
    4100 if self.columns.nlevels > 1:
    4101     return self._getitem_multilevel(key)
-> 4102 indexer = self.columns.get_loc(key)
    4103 if is_integer(indexer):
    4104     indexer = [indexer]

File ~\AppData\Roaming\Python\Python313\site-packages\pandas\core\indexes\base.p
y:3812, in Index.get_loc(self, key)
    3807 if isinstance(casted_key, slice) or (
    3808     isinstance(casted_key, abc.Iterable)
    3809     and any(isinstance(x, slice) for x in casted_key)
    3810 ):
    3811     raise InvalidIndexError(key)
-> 3812     raise KeyError(key) from err
    3813 except TypeError:
    3814     # If we have a listlike key, _check_indexing_error will raise
    3815     # InvalidIndexError. Otherwise we fall through and re-raise
    3816     # the TypeError.
    3817     self._check_indexing_error(key)

KeyError: 'churn'

```

```

In [7]: #Step 4:Encode Categorical Columns
le=LabelEncoder()                                # Male=1, Female=0
df['Gender'] = le.fit_transform(df['Gender'])      # Month-to-month=1, One year
df['Contract'] = le.fit_transform(df['Contract']) # Yes=1, No=0
df['Churn'] = le.fit_transform(df['Churn'])

```

```
In [8]: plt.figure(figsize=(10,6))                                     #Step 5:Visualize Correlation
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



```
In [9]: X = df[['Gender', 'Age', 'MonthlyCharges', 'Tenure', 'Contract']] #Step 6: S
y = df['Churn']
```

```
In [10]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

```
In [11]: model = RandomForestClassifier() #Step 8:Train Random Forest Classifier
model.fit(X_train, y_train)
```

Out[11]: **RandomForestClassifier**

RandomForestClassifier()

```
In [12]: y_pred = model.predict(X_test) #Step 9: Make Predictions & Evaluate
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
[[24  3]
 [11  2]]
```

	precision	recall	f1-score	support
0	0.69	0.89	0.77	27
1	0.40	0.15	0.22	13
accuracy			0.65	40
macro avg	0.54	0.52	0.50	40
weighted avg	0.59	0.65	0.59	40

```
In [13]: #Eg:Example: Male, 35 yrs, ₹85.00/month, 12 months, Month-to-month contract #S
new_customer = pd.DataFrame({
    'Gender': [1],
```

```
'Age': [35],  
'MonthlyCharges': [85],  
'Tenure': [12],  
'Contract': [1]  
})  
  
prediction = model.predict(new_customer)  
print('Churn Prediction (1=Yes, 0=No):', prediction[0])
```

Churn Prediction (1=Yes, 0=No): 0

In []: